GNASCOR

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
A SEMANTIC RELATIONAL LANGUAGE

ORIOR
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

Gnascor is the language the orior compiler is written in. It treats every host, a processor, a database, a web service or a language model, as a black box that answers yes or no questions, and it reads each answer together with what the answer cost. A question is an address, a predicate asked at it, and a timeout; the answer is one bit, 1 where the predicate holds inside the timeout. The timeout is never written by hand. It is derived from the spread of the host's own costs, measured first with no timeout at all.

Two answers taken side by side make a branch pair, and the pair is a state of a finite state machine whose output on each transition is a four-letter mnemonic. The mnemonics are the language's keywords. Each maps to a term the field already has: failover and failback, load shedding, a circuit breaker, a health probe, the saturation, latency and error signals of production monitoring, and the superstate of a statechart.

The method has one rule. A predicate holds or it does not, and that answer carries no noise; a cost is a measurement and always does. The language decides which candidates hold first and ranks only the survivors by cost, and never adds the two into one score. The ranking reads a difference as real only where it clears the candidates' own spread.

This research paper states the language, maps each of its parts to the field's term with a source, collects the experiments that have run on it, and adds three new measurements: how many bits of a transition each kind of label keeps, how often the derived timeout is beaten on a real host, and which states repeat from run to run. It then states three hypotheses the measurements support and the work left open.

CONTENTS

1 THE LANGUAGE 1
1.1 Introduction 1
1.2 Two faces 1
1.3 The data model 1
1.4 Information is coherence 2
1.5 Keywords 2

2 THE QUERY PROTOCOL 5
2.1 The ask 5
2.2 The baseline 5
2.3 The derived timeout 6
2.4 Gate, then rank 6
2.5 Slicing a chain 7
2.6 Contention 8
2.7 Comparing two branches 8
2.8 One ask, many answers 8
2.9 A reference compiler 9
2.10 An ask that does not return 9
2.11 The record 9

3 THE STATE MACHINE 11
3.1 A Mealy machine over branch pairs 11
3.2 The states 11
3.3 The transitions 12
3.4 Superstates 12
3.5 A label is decided in its situation 13
3.6 The tables read against asks 13

4 MEASUREMENTS 15
4.1 What a label keeps of its transition 15
4.2 A scenario put to a real host 16
4.3 The scenario repeated 16
4.4 Which side is asked first 17
4.5 Checks of the definition 17

5 HYPOTHESES AND OPEN PROBLEMS 19

5.1 H1. Friction tracks depth . . . 19
5.2 H2. Bits repeat, costs vary . . . 19
5.3 H3. A label carries its pair . . . 20
5.4 Open problems . . . 20
5.5 Data and code availability . . . 21

6 TERMS . . . 23
6.1 States . . . 23
6.2 Transitions . . . 23
6.3 The protocol . . . 24
6.4 Information . . . 24

BIBLIOGRAPHY . . . 25

1

THE LANGUAGE

1.1 INTRODUCTION

A compiler for many targets has to know what each target does. The usual answer is to write it down: a driver, a machine description, a cost model per part. Gnascor answers the other way. It asks the part, records what the part answered and what each answer cost, and derives every table it needs from those records.

Every host is reduced to one interface, a question with a yes or no answer put to an address. A processor, a database, a web service and a language model all have addresses, and all of them spend time, compute or tokens to answer. In the terms of software testing each host is a black box behind a test oracle, a component that only says whether a check holds [10]. The language never models what is inside the box. It reads the answers and what they cost.

This chapter states the language and its principle. Chapter 2 states the query protocol as definitions and propositions, Chapter 3 the state machine its keywords name, Chapter 4 the measurements, and Chapter 5 the hypotheses they support and the problems left open.

1.2 TWO FACES

A program has two faces that compile to the same machine code. The high order face writes plain statements: *a equals something; b equals something; evaluate a is identical to b.* The assembly face writes the same program short: `a=0;b=1; x = a==b`. Both compile to two reads, one target and one store.

A block declares its face, and a statement mixes the two only inside an explicit escape, `__gsm__(...)`. This is inline assembly with a stricter rule: the face is checked, and nothing crosses between faces unmarked.

1.3 THE DATA MODEL

The language keeps what it learns in files grouped by stem. Files sharing a stem describe one member, a part or a set, and the suffix says which view of the member a file holds. A member carries only the views it has answers for. Table 1.1 gives each view beside the field's nearest term.

A hardware map holds a general block, valid for any part of a class, and under it a block specific enough to be best on one part and no other. A driver written by hand is the general block only,

suffix	holds	field term
`.ksc`	the language map	language specification
`.krs`	the coherence rules	rule set; instruction selection table
`.kcr`	information at or near its Kolmogorov complexity	compressed form
`.knf`	a measured noise floor	baseline; noise floor
`.kcs`	what rebuilds information	generator; decoder
`.kdm`	the hardware map	machine description
`.kqr`	every question put and what came back	measurement log

Table 1.1: The views of a member. The suffixes are the language's own.

since a person writes it once. Here neither block is written: both are read off the part, and keeping the specific one costs nothing.

The query record, `.kqr`, holds every question put to a member and every answer, with costs, refusals and censored samples marked, and the best path per problem read from those same answers. In the field this is a measurement log.

1.4 INFORMATION IS COHERENCE

The views carry Kolmogorov's name because the language rests on one principle. A description at its Kolmogorov complexity has no redundancy: no part of it can be predicted from the rest [13]. A system at coherence has the same property seen from outside. Its parts agree, none spends effort undoing another, and the friction between them is at its floor. Noise costs and carries nothing; information aligns and carries.

The length of a shortest description cannot be computed. Friction can be measured, in time, compute and latency, on any host that answers at all, and the language takes it as its reading of information content. This is the minimum description length principle read from the side of the machine [19]: the best model of data describes it most briefly, and the best arrangement of a computation runs on the host with least friction. Chapter 5 asks what the measured friction tracks, and finds it closer to Bennett's logical depth [2] than to description length.

The crystal, the floor and the construction set are the three sides of the principle. A crystal is information at its complexity, a floor is the part that carries nothing, and a construction set rebuilds information from its primitives.

1.5 KEYWORDS

The language's keywords are four-letter mnemonics. They name the states of a branch pair and the transitions between them, and each maps to a concept the field already has a word for. Table 1.2 gives the map; Chapter 3 gives the machine, and Chapter 6 the sources.

keyword	what it names	field term
dual	both sides answer inside the timeout	active-active
lead, rite	one side answers	active-passive
void	neither side answers	outage
busy	both answer, both slow	saturation
wait	one side lags	latency
blok	a side refuses	error
gray	a side not yet asked	unknown; Belnap's None
pass, back	the answering side changes over	failover, failback
vent	a saturated pair drops one side	load shedding
fuse	both sides lost at once	circuit breaker tripped
jamm	saturation turns into refusal	gridlock
sprk	from nothing to both	recovery
sync, gray	entered and left by marked labels	superstate
the clock	the stream of states and labels	health status stream

Table 1.2: The keywords beside the field's terms.

2

THE QUERY PROTOCOL

Every statement the language runs reduces to one form,

$$[\,\text{address}\,] \;\to\; (\,\text{predicate}\,) \;\to\; [\,\text{cost}\,] \;\to\; \text{bit}.$$

This chapter states the protocol as definitions and propositions and gives, beside each, what has been measured.

2.1 THE ASK

Definition 2.1 (Ask)**.** An *ask* a is an address and a predicate. Put with no timeout it returns its cost $t(a)$. Put with a timeout β it returns one bit,

$$\text{bit}(a) = [\text{predicate holds}] \cdot [\,t(a) \le \beta\,].$$

A missing address, a false predicate, a timeout, an error and an answer too dear all read o. The bit does not keep them apart; the record of costs does. This is the health probe of service monitoring, which passes when it answers correctly inside its timeout and fails on a slow answer as on a wrong one [14].

The written form is `@address?predicate($bound)`. A pair of asks side by side is a branch pair:

```
left_input    = @system.auth?is_active($5ms)
right_input   = @db.user_session?is_valid($12ms)
branch_result = [left_input, right_input]
```

The figures in the example stand for derived ones. No absolute figure enters a program: a bound written by hand fixes a scale the rest of the language never assumes.

On a Windows workstation the ask is realized with no compiler, runtime or driver between the asker and the memory it asks. It finds memory the asker owns holding and leaves it as it was, reads a written word as that word, reads a word nothing writes as not advancing, and reads a timeout with no clock as past the timeout. The clock is itself found by asking: a word the operating system maps into every process, which advances in steps of half a millisecond to a millisecond.

2.2 THE BASELINE

Definition 2.2 (Baseline)**.** The *baseline* of an ask is the set of its costs measured with no timeout, summarized by its least value m, its spread s, the largest less the least, and its mean.

An ask with no timeout returns a cost and no bit, and every timeout is expressed against the baseline. The language ranks on the least value. A run is lengthened by whatever else the host does and never shortened by it, and under noise of that kind the minimum is the estimator of the true cost with the smallest error [4].

Two rules keep a baseline sound. First, an ask stopped by a watchdog did not report its cost; it reported that the cost was at least the watchdog's limit. Survival analysis calls this a right-censored observation, and a mean taken over censored samples as if they were costs reads low [11]. Second, a baseline goes stale: the host under load now is not the host idle later. A reference ask is put beside each real one and measured under the same conditions, a concurrent control in place of a figure carried forward.

On an NVIDIA RTX 3070 (Ampere, sm_86), chains of 100 steps run 10000 turns over 16 runs read every single-instruction operation at $m = 4.05$ to 4.06 cycles a step, a two-instruction operation (NAND, NOR) at 8.05, and an operation with no instruction at 0. The spread reaches 2.22 cycles a step alone and 2.53 interleaved, and the means, 4.14 to 4.45, sit inside it.

2.3 THE DERIVED TIMEOUT

Definition 2.3 (Derived timeout)**.** From k satisfied runs with largest cost $t_{\max}$ and least $t_{\min}$, on a clock whose least resolvable step is g,

$$\beta = t_{\max} + (t_{\max} - t_{\min}) + g.$$

The count k is not fixed in advance. Runs are doubled until a whole step of them lands under the timeout, and before each cycle a satisfied run is checked against it again. The timeout only grows, since a new run can raise the largest or lower the least and never the reverse.

This has the shape of TCP's retransmission timeout [18],

$$\mathrm{RTO} = \mathrm{SRTT} + \max(G,\ 4 \cdot \mathrm{RTTVAR}),$$

a center, plus a multiple of the spread, with the clock granularity G as a floor. TCP smooths its center and spread over a stream of samples; gnascor takes the extremes of a sample whose size the host decides. Both are estimates from a finite sample, and both can be beaten by a correct answer that arrives late, a *spurious timeout.*

Proposition 2.4. *If the k runs a timeout is derived from and one further run are exchangeable and continuous, the further run exceeds β with probability less than $1/(k+1)$.*

Proof. Under exchangeability each of the $k+1$ runs is equally likely to be the largest, and the further run exceeds $t_{\max}$ with probability $1/(k+1)$. Since $\beta \geq t_{\max} + g > t_{\max}$, exceeding β is a strictly smaller event.

Section 4.3 measures the rate on a real host and Chapter 5 reads it against this bound.

2.4 GATE, THEN RANK

The protocol holds to one rule. A predicate holds or it does not, and that answer carries no noise; a cost is measured and carries noise. Decide which candidates hold first, rank only the survivors, and never add the two into one score.

Definition 2.5 (Gate)**.** A relation r is checked on cases $K = \{k_1, \dots, k_m\}$. A candidate c *survives* when $c(k) = r(k)$ for every $k \in K$.

Proposition 2.6. *The set of survivors does not depend on the order in which the cases are put, and no cost enters it.*

Proof. The survivors are $\{c : \bigwedge_{k \in K} c(k) = r(k)\}$, a conjunction, and conjunction is commutative. No term of it is a cost.

The order sets only the price. Putting first the case the most remaining candidates fail prunes fastest, and the gate is planned this way on the host against arithmetic every computing system agrees about; a target is asked only the cases the plan placed. A badly steered plan costs speed and never a wrong survivor.

Definition 2.7 (Rank)**.** Each survivor c is run T times. With

$$m_c = \min_j t_{c,j}, \qquad s_c = \max_j t_{c,j} - m_c, \qquad s = \max_c s_c,$$

and the survivors ordered by m, the least c_1 is preferred to the next c_2 only where $m_{c_2} - m_{c_1} > s$. Otherwise the survivors are one cost.

In optimization this is a lexicographic objective, feasibility before cost. Its alternative, one score folding correctness and cost together at some exchange rate, was measured on modeled costs and ships a wrong program at 2 of 5 exchange rates.

Over every relation the compiler's ladder of operations holds, the gate's survivors equal those of every candidate asked every case, with none lost: 1,202 of 1,227 candidates for addition, 1,047 of 1,083 for subtraction, 411 of 698 and 408 of 559 for the two shifts, 0 of 8 for equality. Every impostor the ladder admits, 507 in all, fails at the first or second case placed. On the RTX 3070, of the 2,927 instruction forms in the part's machine description put in the place of a loop's backward branch, 8 answer every case, every one a branch instruction; the cheapest two differ by 0.0525 ns a turn against a spread of 0.9260, and the rank reads them as one cost.

2.5 SLICING A CHAIN

A chain of links returns one cost, and one number names no link.

Definition 2.8 (Known order)**.** For a chain of n links with $n + 1$ a power of two, ask r covers link c where $(r + 1) \,\&\, (c + 1)$ has an odd count of ones. The covering matrix S is a Hadamard matrix of order $n + 1$ with its first row and column dropped, read as 0 and 1.

Proposition 2.9. *With $b = Sx$ the costs returned and x the per-link costs,*

$$(n + 1)\, x = 4\, S^{\mathsf{T}} b - 2 \left(\sum b \right) \mathbf{1}.$$

Proof. Each link is covered by $(n + 1)/2$ asks and any two by $(n + 1)/4$, hence $S^{\mathsf{T}} S = \frac{n+1}{4}(I + J)$ [21]. Then $4S^{\mathsf{T}} b = (n + 1)\, x + (n + 1)\big(\sum x\big)\mathbf{1}$, and $\sum b = \frac{n+1}{2} \sum x$ gives the second term.

The solve is integer additions and one division by a power of two. This is a weighing design. Hotelling showed that weighing objects in combinations, in place of one at a time, cuts the error of every weight [9], and the 0 and 1 form is the S-matrix design of Hadamard transform spectrometry, for a detector that can only add [21]. Against one ask a link the error falls by about $\sqrt{n+1}/2$.

The identity holds entry by entry at 3, 7, 15, 31 and 63 links, and integer costs are recovered exactly. In simulation the gain is 1.09 at 3 links, 2.07 at 15 and 8.16 at 255, against 1.00, 2.00 and 8.00 predicted; against a randomly drawn covering the known order wins by 5.5 times at the median worst-link error and 125.1 times at the worst of 400 draws. Put to the host, every neighboring pair of seven links reads in its true order in 44 to 54 of 64 passes, past twice the spread.

The obvious alternative fails. Reading a link as the difference of two neighboring prefix costs lands two measurements' noise on one link: 1.43 floors of noise against a signal of 1.00, and 56% of link pairs ordered correctly where a coin orders 50%.

2.6 CONTENTION

Where links contend, a set costs more than the sum of its parts. Let N sweep asks cover u links each, with y what remains of an ask's cost once the known order's links are taken out, and model

$$y = \alpha + \gamma u + \delta u^2 + \varepsilon.$$

Contention is $\delta > 0$. With $C_{pq} = N \sum pq - \sum p \sum q$, $s = u^2$, and the linear term taken out,

$$P_{ss} = C_{uu}C_{ss} - C_{us}^2, \quad P_{sy} = C_{uu}C_{sy} - C_{us}C_{uy}, \quad P_{yy} = C_{uu}C_{yy} - C_{uy}^2,$$

the quadratic slope stands above twice its standard error exactly where

$$P_{sy}^2 \,(N+1) > 4\, P_{yy}\, P_{ss},$$

a test in exact integers with no division.

In simulation the test finds contention in 3.0% of trials with none, 83.6% with up to 80 units a pair, and 100% with up to 200. On the RTX 3070, two dependency chains sharing no link, interleaved a step at a time, cost the dearer of the two alone in all 91 pairs of operations that cost a cycle a step or more. Chains that share no link do not contend.

2.7 COMPARING TWO BRANCHES

Two branches over one relation are compared link by link through the signed difference at each link. A total cannot say which branch holds a link: the total is unchanged when the branches trade places, and the question changes sign. The sign at a link reads lead or rite, a difference inside the floor reads dual, a link neither branch passes reads void, and pass marks where the sign changes along the chain. A path that changes branch at a pass pays for the change as a link of its own. This comparison is stated and not yet measured.

2.8 ONE ASK, MANY ANSWERS

An answer is 128 bits wide. It can carry one bit for each of 128 checks, or one bit for a set, 1 where every check in it holds and the set halved on a 0. The second is group testing [6], and it pays only

where checks seldom fail: past a failing share of $(3 - \sqrt{5})/2$, about 38%, testing one at a time costs no more than any pooled scheme [22].

Against the cases the gate placed, one bit a check takes the fewest asks on every relation. For addition, 1,227 checks with 25 failing take 10 asks at a bit a check, 129 pooled and halved, and 299 pooled in a drawn order. For a shift where 38.5% fail, pooling takes 1,593 asks where one check an ask takes 1,396, as the cutoff predicts.

2.9 A REFERENCE COMPILER

A program written in C and in the part's own instructions must answer as the vendor compiler's code for the C does on every lane, and cost no more. This is differential testing [15]: two implementations of one specification run on the same inputs, every disagreement a defect in one of them. Where the language's code costs more, the forms the vendor compiler wrote name the next questions to put to the part. The vendor listing is a reading of what the part can do and never a rule: a form enters a rule set once the part answers for it.

Over 98 instructions the vendor compiler wrote in 18 blocks, the language's disassembler gives the vendor's text and its assembler the vendor's operation bits wherever the machine description holds the form. The vendor compiler writes NAND and NOR as two instructions at 8.05 cycles a step where one three-input logic instruction computes either.

2.10 AN ASK THAT DOES NOT RETURN

A timeout counts as a reading of 0 only where the run returns. A run that never returns ends in a reset by the display watchdog, and a reset is not a reading. A body loops without end only where control reaches a line it has passed and the way back never closes. An ask that holds a waited read therefore carries its own bound inside the body, a turn count kept in registers the bits under test cannot reach. This is a loop variant, a counter that strictly falls and guarantees termination [7], placed where no fault under test can disable it.

2.11 THE RECORD

A known order is a carrier and a shuffle is not: the same asks reach the part either way, and only a record of the order makes the answers decodable. The record keeps what the host cannot reproduce, the target's answers, their costs, refusals and censored samples, and the seeds the rest follows from.

A reading answers for the part it was taken on and the size it was taken at, and for nothing else by default. On modeled costs the arrangement winning at size 3 costs 79.4 times the best at size 300, a winner carried to another part costs 3.06 times that part's best, and the two penalties multiply.

Two parts whose readings agree inside a floor do not form a group by that alone. Agreement asked from one part's floor is not symmetric, and at the coarser floor it is not transitive: sm_86 agrees with sm_87 and sm_87 with sm_89, and sm_86 does not agree with sm_89. Agreement

inside a tolerance is not an equivalence relation, and a group needs a rule that names its representative.

3

THE STATE MACHINE

3.1 A MEALY MACHINE OVER BRANCH PAIRS

Two asks put side by side form a branch pair. Each cycle the pair is read as a state, and each change of state from cycle $N - 1$ to cycle N is given a label. A machine whose output is written on its transitions, a function of the state left and the input that moved it, is a Mealy machine [16]. The language's clock is that output: a stream of states and labels from which the health of a system can be read, as from a status page.

3.2 THE STATES

Four states come from the two bits alone, and four from what the bits were measured with.

state	reading	field term
dual	1, 1	both sides healthy; active-active
lead	1, 0	left serves; active-passive
rite	0, 1	right serves; active-passive
void	0, 0	outage
busy	both satisfied, both dear	saturation
wait	one satisfied, one past the timeout	latency
blok	a side ended its asker	error
gray	a side not asked	unknown; Belnap's None

Table 3.1: The base states.

busy, wait and blok are three of the four golden signals of production monitoring: saturation, latency and errors [3]. The fourth, traffic, is the rate of asks, which the protocol sets and does not read.

gray is no answer, and it is not void, which is an answer of no. Belnap's four-valued logic for a machine that answers questions has a value for exactly this, None, told nothing, beside True, False and Both [1]. In his order of information None sits below every other value: it is compatible with each of them until an ask is put, the language's "every state at once". Entering gray is the loss of the ability to ask, labeled fuzz, and leaving it is a first observation, labeled fizz.

Definition 3.1 (Reading a state off asks)**.** A side reads 1 where its ask satisfied inside the timeout. A pair reads blok where a side ended its asker, gray where a side was not asked, wait where one

side satisfied and the other satisfied past the timeout, and busy where both satisfied and each cost more than every satisfied ask of the baseline. The edge busy is read against is the baseline's largest satisfied cost, never a figure written in.

The busy edge is the rank rule of Chapter 2 applied to one side: a cost past the baseline's least by more than the baseline's spread is past its largest.

3.3 THE TRANSITIONS

Table 3.2 is the language's first transition table, the label for each pair of past and present state.

past	dual	lead	rite	void	busy	wait	blok	gray
dual	nexus	drop	sync	drop	sync	sync	halt	fuzz
lead	join	core	pass	drop	sync	sync	halt	fuzz
rite	join	back	surv	drop	sync	sync	halt	fuzz
void	sprk	wake	wake	zero	sync	sync	halt	fuzz
busy	sync	sync	sync	drop	drag	sync	jamm	fuzz
wait	sync	sync	sync	loss	sync	hold	halt	fuzz
blok	sync	sync	sync	drop	sync	sync	dead	fuzz
gray	fizz	fizz	fizz	fizz	fizz	fizz	fizz	–

Table 3.2: The state transition table: rows are the state at cycle $N - 1$, columns the state at cycle N.

The diagonal, nexus, core, surv, zero, drag, hold and dead, is the steady states. pass and back are the handoff between the two sides, failover and failback. drop and jamm are collapses: a stable state lost, and saturation turned into refusal, which a distributed system knows as gridlock. halt is any state reaching a refusal.

A second table splits the transitions out of high load (Table 3.3).

past	dual	lead	rite	void
busy	surg	vent	vent	drop
wait	snap	lead	rite	loss
dual	nexus	tilt	tilt	fuse

Table 3.3: The high load transition table.

surg is saturation clearing into a burst of throughput. vent is a saturated pair dropping one side, load shedding. snap is a lagging side catching up. tilt is dual decaying to one side. fuse is both sides lost at once with no decay: the breaker trips. The language leaves fuse, jamm and halt without a recovery path; the field's answer is the circuit breaker, which opens on failure, stops sending, and after a wait lets one trial request through before it closes again [17]. sprk, from void straight to dual, is that trial succeeding.

3.4 SUPERSTATES

sync covers 22 cells of Table 3.2. A label that covers many transitions loses them: a branch that reads only sync cannot tell dual to busy from blok to lead. The language's answer is to make sync

a superstate. A transition the table marks sync is written as a passage through it, entering by fuzz carrying the state it left and leaving by fizz carrying the state it reaches:

$$[\text{dual}] \xrightarrow{\text{fuzz+dual}} [\text{sync}] \xrightarrow{\text{fizz+busy}} [\text{busy}].$$

gray is entered and left the same way. This is Harel's superstate [8]: one state that stands for a set of others, entered and left through labeled transitions. Here the labels carry the identity of the transition, and the pair is read back from them exactly.

Proposition 3.2. *A trip through gray is atomic: a fuzz opens it, one fizz closes it, and no fuzz opens inside another.*

Proof. fizz labels only transitions out of gray to another state, fuzz only transitions into gray from another state, and gray to gray carries no label. Between a fuzz and the next fizz the pair is gray throughout, and no transition out of gray is a fuzz.

Over 500 random traces of 40 cycles, every side drawn from satisfied, heavy, not satisfied, late, ended and unasked, every gray trip is atomic, and the exact pair of all 3,035 sync passages is read back from their labels alone.

3.5 A LABEL IS DECIDED IN ITS SITUATION

Where both tables name one transition they can disagree. dual to lead is drop in the first table and tilt in the second; dual to void is drop in the first and fuse in the second. The language does not pick one for every case. Two names for one pair are two candidates, and the asks that read the transition choose between them the way the gate chooses an arrangement: a bit excludes what does not hold, and a magnitude ranks the survivors.

The protocol already carries what separates them. A side reading 0 carries why: past its timeout, not satisfied, or ended. dual to lead is tilt where the right side came in past its timeout by a fraction of a cycle, and drop where it did not answer at all or came in late by a timeout's measure. dual to void is fuse where both sides go at once with no decay, and drop where the collapse ran through a timeout. Where the line between a fraction and a timeout falls is read off the baseline and never written in. Under this rule every pair the tables name takes a label from its own candidates in each of the 256 situations two sides can be in.

3.6 THE TABLES READ AGAINST ASKS

Read against real asks, the tables raise six points of definition, each left for the language's design to settle.

- tilt and wait name one event. Read off asks, a satisfied side beside one a fraction late is wait, and dual to lead by tilt never arrives.
- A name stands for a state and for a transition. The second table labels wait to lead as lead; core is the pair 1, 0 in the branch pair table and lead to lead in the transition table.

- void does not separate two sides both past the timeout from two that both did not hold. The kind each ask carries does, and the record keeps it.
- Two prose rules carry exceptions the tables hold: any state to blok is halt, except busy to blok (jamm) and blok to blok (dead); any stable state to void is drop, except dual to void where both sides go at once (fuse). The tables are taken as the definition.
- dual at maximum is called unstable, and dual to dual, nexus, is listed as a steady state.
- shift, in the branch pair table, has five letters where every other keyword has four.

4

MEASUREMENTS

Three new measurements are reported here: how much of a transition each form of label keeps, how the state machine behaves when a fixed scenario is put to a real host many times, and whether the side a branch is read from leaves a mark. The transition tables are read directly from the language's definition for each measurement, never transcribed.

4.1 WHAT A LABEL KEEPS OF ITS TRANSITION

Method. A transition is its pair, the state left and the state reached. A branch that reads only the label recovers the pair exactly where the label is reached from one pair, and loses information where it is shared. The loss is the conditional entropy $H(\text{pair} \mid \text{label})$ [20], and the label keeps $H(\text{pair}) - H(\text{pair} \mid \text{label})$, the mutual information between the two. The 63 named cells of Table 3.2 are weighted alike, $H(\text{pair}) = 5.977$ bits. Each pair is put through every situation a label is decided in, four kinds of answer at four costs for each side, against a timeout of 1000 and an edge of 120 units, with every situation weighted alike within its pair.

Results. Table 4.1 gives four forms of label.

form of label	labels	$H(\text{pair} \mid \text{label})$	kept
one name a cell, as the tables print it	19	2.675	3.302
one name, decided in its situation	26	2.129	3.848
sync written as its passage	41	1.121	4.856
gray trips also carrying their state	53	0.498	5.480

Table 4.1: Bits of a transition each form of label keeps, of 5.977.

Read from the tables alone, sync is reached from 22 pairs and leaves 4.459 bits of its pair unread; fizz and fuzz, 7 pairs each, leave 2.807; drop, 6 pairs, 2.585; halt, 5 pairs, 2.322; join and wake, 2 pairs each, 1.000.

Deciding each label in its situation moves six of sync's pairs to the high load table's names, surg, vent twice, snap, lead and rite, and leaves sync reached from 16 pairs at 3.999 bits. Writing sync as its passage, fuzz carrying the state left and fizz the state reached, removes its loss entirely.

What remains, 1.121 bits, sits in eight labels: fizz and fuzz into and out of gray, 2.807 bits each; drop, 2.539; halt, 2.322; join, tilt, vent and wake, 1.000 each. A gray trip loses what a sync passage keeps because its labels carry no state. Letting fuzz into gray carry the state it left, and fizz out of gray the state it reaches, raises what a label keeps to 5.480 bits of 5.977. The loss left, 0.498 bits,

sits in drop, halt, join, tilt, vent and wake. drop and halt are the largest: each is a rule, any stable state to void and any state to blok, and a rule covering many pairs names none of them.

4.2 A SCENARIO PUT TO A REAL HOST

Method. A scenario fixes, for each of ten cycles, what each side is: satisfied, not satisfied, late, or not asked. Each side is realized as a run of a million asks between two readings of the host clock, after four baseline cycles from which the timeout and the busy edge are derived. The host is a Windows workstation.

Result. The clock printed by one run is

```
[dual] -(drop)-> [lead] -(pass)-> [rite] -(join)-> [dual] -(fuzz+dual)-> [sync]
-(fizz+wait)-> [wait] -(loss)-> [void] -(fuzz)-> [gray] -(fizz)-> [dual] -(fuse)-> [void]
-(sprk)-> [dual]
```

In the field's terms: both sides healthy, the right side lost, failover to the right, both healthy again, the left side lagging, the lagging side timing out into an outage, the left side going unobserved and returning, the breaker tripping, and recovery. Every state agrees with what the recorded sides imply.

4.3 THE SCENARIO REPEATED

Method. The scenario is put 40 times in succession, about 20 seconds in all, and every trace is read back. Two batches of 40, A and B, were put minutes apart.

Results. Table 4.2.

reading	A	B
cycles read as other than the recorded sides imply	0 of 400	0 of 400
satisfied asks past the timeout their run derived	2 of 480	0 of 480
both-satisfied cycles read busy in place of dual	8 of 160	15 of 160
distinct clocks	5	8
runs printing the most common clock	32 of 40	28 of 40
derived timeout, least	72,182	79,782
derived timeout, median	150,810	167,807
derived timeout, most	191,351	212,558

Table 4.2: Two batches of forty runs of the scenario. Timeouts are in counts of the host clock.

The machine reads its record exactly. In all 800 cycles the state is one the recorded sides imply. Every departure from the scenario is the host's, in timing.

Spurious timeouts. Two satisfied asks of 960, 0.21%, came in past the timeout their run derived: correct answers, late. Both were in batch A, both in the cycle where a satisfied side stands beside an unasked one and the pair reads gray whatever the satisfied side does, and both in runs whose derived timeouts were among the lowest, 72,910 and 74,596 against a median of 150,810.

The timeout varies. Within a batch the largest derived timeout is 2.65 times the least in A and 2.66 times in B. Each run derives its own from the host as it is at that moment, and the spread across runs measures how far a figure written once would have been wrong.

busy varies; nothing else does. Of 320 both-satisfied cycles, 23 read busy, 5% in A and 9.4% in B. The scenario does not choose busy; the host's load does. The distinct clocks differ in these readings only. In A each of the four both-satisfied cycles reads busy in 2 runs and dual in 38; in B in 3, 2, 6 and 4 runs. Every other cycle reads the same state in all 80 runs: lead, rite, void, wait and gray, each decided by which sides satisfied, never varied.

busy reads the host instead of a side. In batch B, 53 of the 320 sides of the both-satisfied cycles came in past the edge, a share of 0.1656. Were the two sides independent, both would be past it at once in $160 \times 0.1656^2 = 4.39$ cycles. They were past it together in 15, 3.4 times as often.

4.4 WHICH SIDE IS ASKED FIRST

Method. core and surv, lead satisfied and rite satisfied, are distinct only where the side a branch is read from leaves a mark. Two branches doing the same work are put through the known order of Proposition 2.9, the branch asked first alternating from trial to trial, and every pass is solved in exact integers.

Results. On the host the branch asked first reads dearer in 12 of 29 trials that are not even, with no lean past twice the spread. With one branch made 512 reads dearer, the cheaper reads cheaper in 46 to 58 of 64 comparisons at every link, from both sides alike. On this host a steady lead and a steady rite are one state seen from two sides; on a part whose side does leave a mark they are two.

4.5 CHECKS OF THE DEFINITION

On traces whose answers are known, every rule of Chapter 3 holds. Every pair the tables name takes a label from its own candidates in all 256 situations; gray trips are atomic over 500 random traces of 40 cycles; and all 3,035 sync passages in those traces read back their exact pair from their labels. Of the 49 transitions among the first seven states, 12 take a name nothing else uses and sync covers 22. Where both tables name a transition, 3 agree, the second names 7 the first calls sync, and 2 disagree.

5

HYPOTHESES AND OPEN PROBLEMS

Three hypotheses follow from the measurements. Each is stated with the evidence for it, a prediction, and the observation that would refute it.

5.1 H1. FRICTION TRACKS DEPTH

Hypothesis. The cost the protocol measures of a computation tracks the length of its longest chain of dependent steps, and not the number of its steps.

Evidence. On the RTX 3070 an operation of no instruction costs 0 cycles a step, of one instruction 4.05, and of two dependent instructions 8.05: cost grows by one instruction latency per link of the chain. Two chains that share no link, interleaved a step at a time, cost the dearer of the two alone in all 91 pairs measured. The interleaved cost is the larger of the two. In scheduling this is the critical path [12]: the duration of a set of tasks is the length of its longest dependent sequence, however many tasks run beside it.

Reading. The language takes friction as its measure of information (Chapter 1). If H1 holds, what it measures is not the length of a shortest description but something closer to Bennett's logical depth, the time a near-shortest program takes to produce an object [2]. The two differ sharply. A random string has maximal Kolmogorov complexity and almost no depth, and Bennett's own examples of shallow objects are random sequences and perfect crystals. A computation at its Kolmogorov complexity can still be cheap to run, and the protocol would read it as cheap. In measurable form, the principle *information is coherence* reads as *cost is depth*.

Prediction. A dependent chain of d one-instruction operations costs about $4.05\,d$ cycles a step on this part, whatever independent work is interleaved with it, until the interleaved work contends for a shared unit.

Refutation. Independent chains whose interleaved cost is the sum of their costs, or a dependent chain cheaper than its depth times the instruction latency.

5.2 H2. BITS REPEAT, COSTS VARY

Hypothesis. The states of the machine fall into two classes. States decided by which sides satisfied (lead, rite, void, wait, gray, blok) are reproducible exactly from run to run. States decided by cost (busy against dual, tilt against drop, fuse against drop) vary with the host's load and are reproducible only as rates. The gate and rank rule of Chapter 2 holds inside the state alphabet itself.

Evidence. Over 80 runs of one scenario every bit-decided state repeated in every run, and every difference between runs was a both-satisfied cycle reading busy in place of dual, at 5% in one batch and 9.4% in the other (Section 4.3). Both sides of a pair came in past the busy edge together 3.4 times as often as independence predicts. A slow moment of the host slows both sides at once. busy reads load common to the pair, the common-mode noise of a measurement that shares one host, and not a property of either address.

Consequence. A program should branch on bit-decided states and treat cost-decided states as signals with a rate. busy should not be attributed to either address of a pair that shares a host. Production monitoring draws the same line: saturation is a property of a shared resource, and it is watched as a trend and not acted on per request [3].

The spurious timeout. By Proposition 2.4 a timeout derived from k exchangeable runs is beaten by a further run with probability below $1/(k+1)$. In batch B the timeouts were derived from 64 or 128 runs, a bound of 1.5%. The measured rate over both batches is 0.21%, well inside it. Both late answers fell in runs whose timeouts were among the lowest, derived in a quiet moment of the host. There the runs the timeout was derived from and the run that beat it were not exchangeable: the host had changed between them. This is the tail latency of large services [5] seen on one machine, and it is why a reference ask is put beside each real one in place of a timeout carried forward.

Prediction. Two asks on separate hosts pass the busy edge together at about the product of their separate rates.

Refutation. A bit-decided state that differs between runs whose recorded sides agree; or, on separate hosts, joint passes far above the product.

5.3 H3. A LABEL CARRIES ITS PAIR

Hypothesis. A transition label loses nothing a branch can use where it is decided in its situation and every passage through a superstate carries the state left and the state reached.

Evidence. Table 4.1. The labels as printed keep 3.302 of 5.977 bits of the transition. Deciding them in their situation keeps 3.848; writing sync as its passage, 4.856; letting gray trips carry their state as well, 5.480. The 0.498 bits still lost sit in labels that each name a rule covering many pairs, drop and halt the largest.

Consequence. The superstate form already used for sync extends to gray at no cost to the atomicity of Proposition 3.2, since only the labels change and not where they fall. The rule labels, drop, halt, join, tilt, vent and wake, can either carry their pair as sync does or stay as names; that is a choice of the language's design and the measurement only prices it.

Refutation. A pair that cannot be recovered from a label carrying it, which would mean two transitions share a pair and the machine is not a function of its pairs.

5.4 OPEN PROBLEMS

1. **What the baseline is per.** A timeout is expressed against the baseline, and the baseline has no keying. Per host is coarse enough to carry signal and too coarse to remember a preference part by part. Per part and operation splits the samples until each is noise. Chaining clears the noise floor, and the chain length that makes a per-part baseline usable is not measured.

2. **Whether a handoff moves the channel.** Whether pass should make the right address the primary channel is undecided. In the field, failover usually does and failback is made a deliberate step.

3. **Recovery from collapse.** jamm, halt and fuse are named and have no defined recovery. The circuit breaker [17] gives one: stop asking for a waiting period, then let one trial ask through, closing on success and opening again on failure. In this language the trial succeeding is sprk.

4. **A loop construct.** The language has no syntax for a query loop. A loop has to express: a pair put each cycle; the timeout left implicit and derived, the unbound pass first; a branch on the label with its pair carried; a turn count that bounds the loop where a fault cannot reach it; and the record written as it runs. The field's nearest form is the control loop of a reconciling controller. The syntax is the language's to define.

5. **A difference inside the floor.** Whether a link whose difference between two branches sits inside the floor reads dual, both alike, or gray, neither known to be cheaper.

6. **Comparing branches.** The link-by-link comparison of two branches is stated and not yet measured.

5.5 DATA AND CODE AVAILABILITY

The language and the programs that produce every figure in this research paper are in the orior repository, `https://github.com/dstroy0/orior`. Each program writes the traces it reads.

6

TERMS

The language's keywords are its own and are kept. Each entry below gives the field's term for the same concept and its source. Terms with no keyword are listed where this research paper uses them in a narrower sense than the field does.

6.1 STATES

dual. Both sides answer inside the timeout. Active-active operation of a redundant pair.
lead, rite. One side answers, the left or the right. Active-passive operation, the serving side named.
void. Neither side answers. An outage; in a probe, both probes failed.
busy. Both sides answer, each dearer than every satisfied ask of the baseline. Saturation, one of the four golden signals of monitoring [3].
wait. One side answers and the other answers past the timeout. Latency, the second golden signal [3].
blok. A side refuses or ends its asker. Errors, the third golden signal [3].
gray. A side not asked: no answer, as against void, an answer of no. Belnap's value None, told nothing, in his four-valued logic for a question-answering machine [1].
sync. A superstate standing for every transition between a load state and a binary one, entered by fuzz carrying the state left and left by fizz carrying the state reached. Harel's superstate [8].

6.2 TRANSITIONS

Mnemonic, label. The output written on a transition of a Mealy machine, a function of the state left and the input that moved it [16].
nexus, core, surv, zero, drag, hold, dead. Self-loops: a state kept from one cycle to the next.
pass, back. Failover and failback, the serving side changing from left to right and back.
drop. A stable state lost to void.
jamm. Saturation turning into refusal. Gridlock.
halt. Any state reaching a refusal.
loss. A lagging side timing out into an outage.
sprk. From void straight to dual. Recovery; the trial request of a half-open circuit breaker succeeding [17].
wake, join. One side returning from void; a second side joining one already serving.

surg, snap. Saturation clearing into full service; a lagging side catching up.
vent. A saturated pair dropping one side. Load shedding.
tilt. Full service decaying to one side by a fraction of a cycle's lag.
fuse. Both sides lost at once with no decay. A tripped circuit breaker [17].
fuzz, fizz. Entry into and exit from a superstate, gray or sync.
Coherence clock. The stream of states and labels. A health status stream.

6.3 THE PROTOCOL

Ask. An address and a predicate, returning a cost unbound and a bit bound. A health probe with a timeout [14].
Unbound pass, baseline. Costs measured with no timeout and their least, spread and mean. A benchmark baseline, ranked on its minimum [4].
Derived timeout. The timeout read off the baseline. An adaptive timeout, of the form of TCP's retransmission timeout [18].
Watchdog stop. An ask stopped at a limit before it reported its cost. A right-censored observation [11].
Spurious timeout. A correct answer arriving past the timeout.
Gate, rank. Feasibility decided on exact predicates, then cost ordered among the feasible. A lexicographic objective; filter then rank.
Known order, slicing. Asks over covering sets of links in the order of an S-matrix. A Hadamard weighing design [9, 21].
Contention. A set of links costing more than the sum of its parts. A quadratic interaction term.
Pooled ask. One bit for a set of checks. Group testing [6, 22].
Reference compiler. Holding the language's code against a vendor compiler's on every input. Differential testing [15].
Turn count. A counter kept inside a loop body that bounds it. A loop variant [7].
Floor. The noise floor of a measurement, written to the language's `.knf` view.

6.4 INFORMATION

Information is coherence. The language's principle that friction measures information content. Kolmogorov complexity [13] read through the minimum description length principle [19]; by H1, closer to logical depth [2].
Kept bits. $H(\text{pair}) - H(\text{pair} \mid \text{label})$, the mutual information between a transition and its label [20].
Critical path. The longest chain of dependent steps [12].

BIBLIOGRAPHY

[1] N. D. Belnap. *A Useful Four-Valued Logic*. In J. M. Dunn and G. Epstein, eds., Modern Uses of Multiple-Valued Logic, Reidel, Dordrecht, 5-37, doi:10.1007/978-94-010-1161-7_2. 1977.

[2] C. H. Bennett. *Logical Depth and Physical Complexity*. In R. Herken, ed., The Universal Turing Machine: A Half-Century Survey, Oxford University Press, 227-257. 1988.

[3] B. Beyer, C. Jones, J. Petoff, N. R. Murphy, eds. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, chapter 6, Monitoring Distributed Systems. 2016.

[4] J. Chen, J. Revels. *Robust Benchmarking in Noisy Environments*. arXiv:1608.04295. 2016.

[5] J. Dean, L. A. Barroso. *The Tail at Scale*. Communications of the ACM 56(2) 74-80, doi:10.1145/2408776.2408794. 2013.

[6] R. Dorfman. *The Detection of Defective Members of Large Populations*. Annals of Mathematical Statistics 14(4) 436-440. 1943.

[7] R. W. Floyd. *Assigning Meanings to Programs*. Proceedings of Symposia in Applied Mathematics 19, Mathematical Aspects of Computer Science, American Mathematical Society, 19-32. 1967.

[8] D. Harel. *Statecharts: A Visual Formalism for Complex Systems*. Science of Computer Programming 8(3) 231-274. 1987.

[9] H. Hotelling. *Some Improvements in Weighing and Other Experimental Techniques*. Annals of Mathematical Statistics 15(3) 297-306. 1944.

[10] W. E. Howden. *Theoretical and Empirical Studies of Program Testing*. IEEE Transactions on Software Engineering SE-4(4) 293-298, doi:10.1109/TSE.1978.231514. 1978.

[11] E. L. Kaplan, P. Meier. *Nonparametric Estimation from Incomplete Observations*. Journal of the American Statistical Association 53(282) 457-481. 1958.

[12] J. E. Kelley, M. R. Walker. *Critical-Path Planning and Scheduling*. Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference, 160-173. 1959.

[13] A. N. Kolmogorov. *Three Approaches to the Quantitative Definition of Information*. Problems of Information Transmission 1(1) 1-7. 1965.

[14] The Kubernetes Authors. *Configure Liveness, Readiness and Startup Probes*. Kubernetes documentation, https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/.

[15] W. M. McKeeman. *Differential Testing for Software*. Digital Technical Journal 10(1) 100-107. 1998.

[16] G. H. Mealy. *A Method for Synthesizing Sequential Circuits*. Bell System Technical Journal 34(5) 1045-1079. 1955.

[17] M. T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Stability Patterns, Circuit Breaker. 2007.

[18] V. Paxson, M. Allman, J. Chu, M. Sargent. *Computing TCP's Retransmission Timer*. RFC 6298, Internet Engineering Task Force, doi:10.17487/RFC6298. 2011.

[19] J. Rissanen. *Modeling by Shortest Data Description*. Automatica 14(5) 465-471, doi:10.1016/0005-1098(78)90005-5. 1978.

[20] C. E. Shannon. *A Mathematical Theory of Communication*. Bell System Technical Journal 27, 379-423 and 623-656. 1948.

[21] N. J. A. Sloane, M. Harwit. *Masks for Hadamard Transform Optics, and Weighing Designs*. Applied Optics 15(1) 107-114. 1976.

[22] P. Ungar. *The Cutoff Point for Group Testing*. Communications on Pure and Applied Mathematics 13(1) 49-54. 1960.